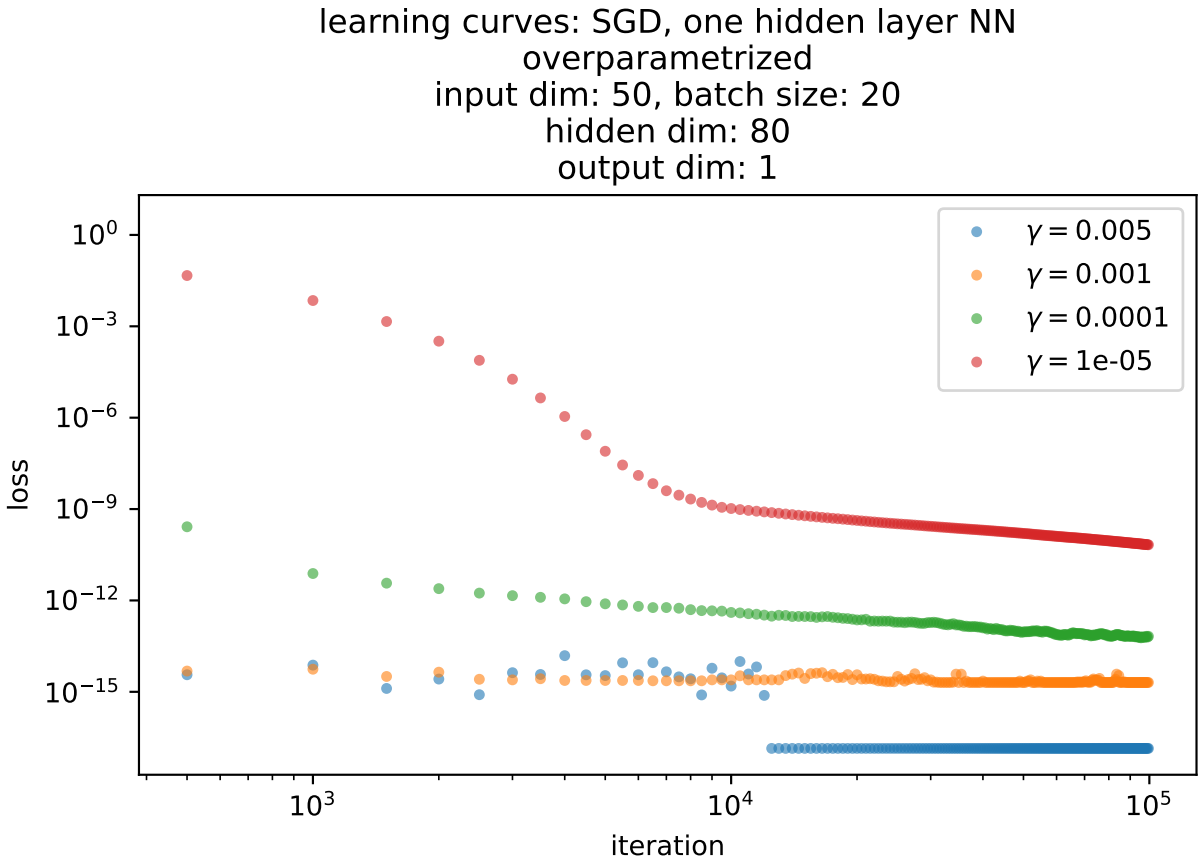
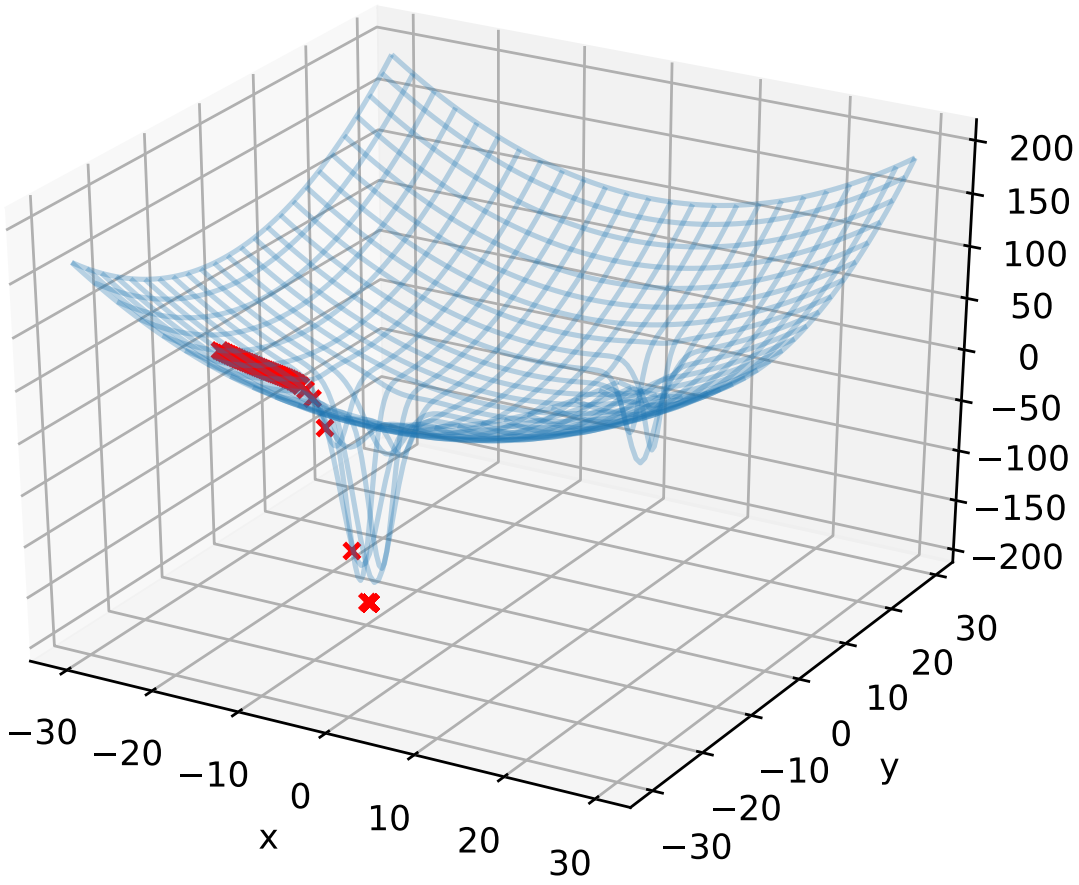
Machine learning Tek 5: neural networks




200
150
100
50
0
−50
−100
−150
−200
−30
−20
−10
0
10
20
30
x
30
20
10
0
−10
−20
−30
y

Vectors, matrices, neural nets

Derivation, gradient descent, backpropagation

Training and visualizing some neural nets

Other techniques

MNIST

Overparametrization and underparametrization

Speed and profiling

Ressources

- https://www.deeplearningbook.org/
- https://course.fast.ai/
- https://d2l.ai/
- https://mlelarge.github.io/dataflowr-web/dldiy_ens.html
- https://playground.tensorflow.org/
- http://www.jzliu.net/blog/simple-python-library-visualize-neural-network/

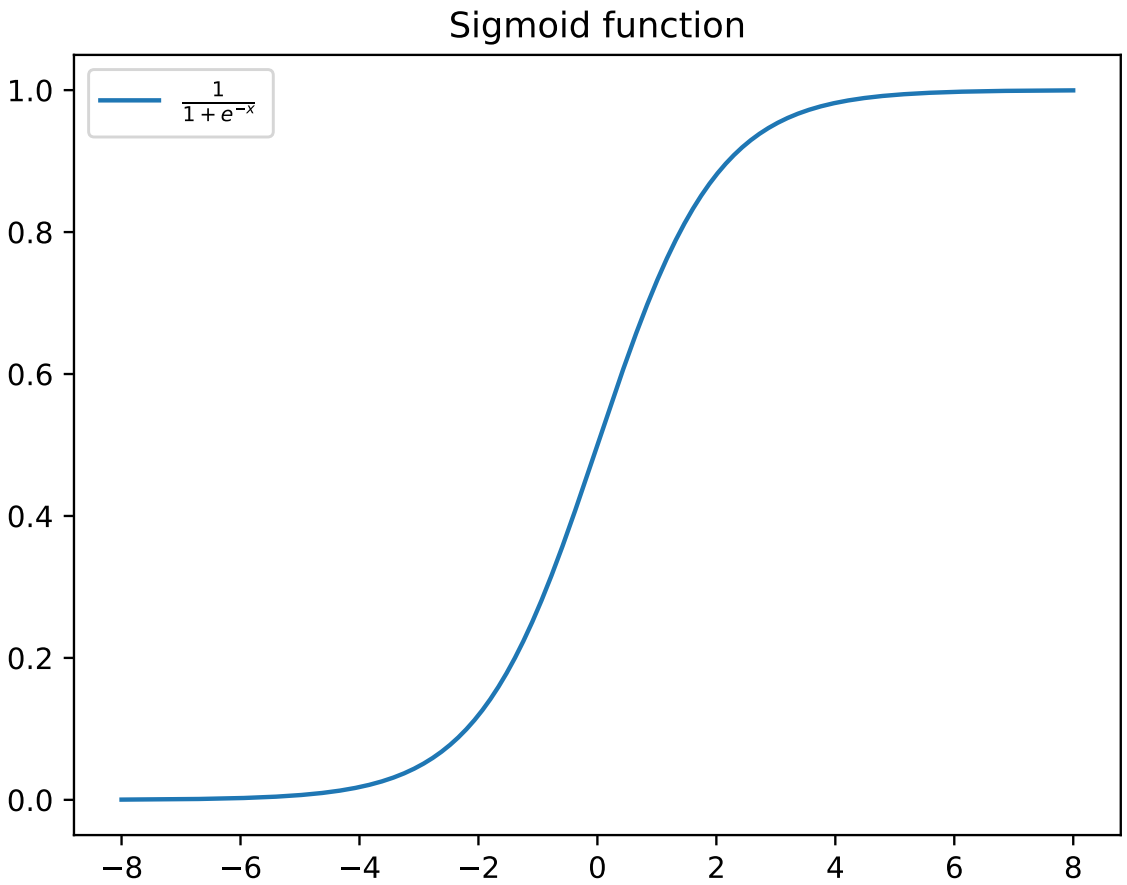
Elementary neuron

A **neuron** is a **mapping** from a multidimensional input $x = (x_1, ..., x_d)$ to a real number.
This function depends on **parameters** called the **weights** $w = (w_1, ..., w_d)$ and the **intercept** b (omitted in the following slides to simplify the notations).

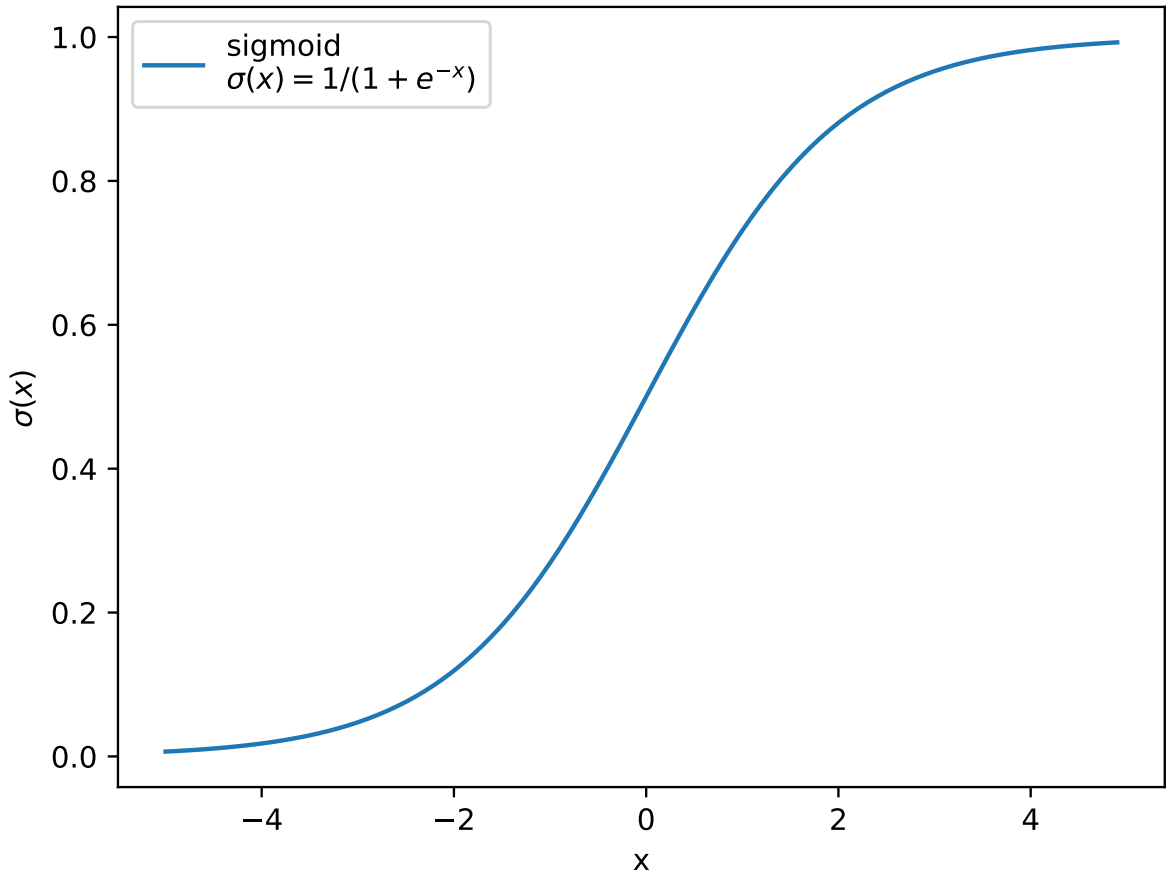
$$f(x) = \sigma(\sum_{i=1}^{d} x_i w_i) \tag{1}$$

Where σ is a non linear function, for instance a **sigmoid**.

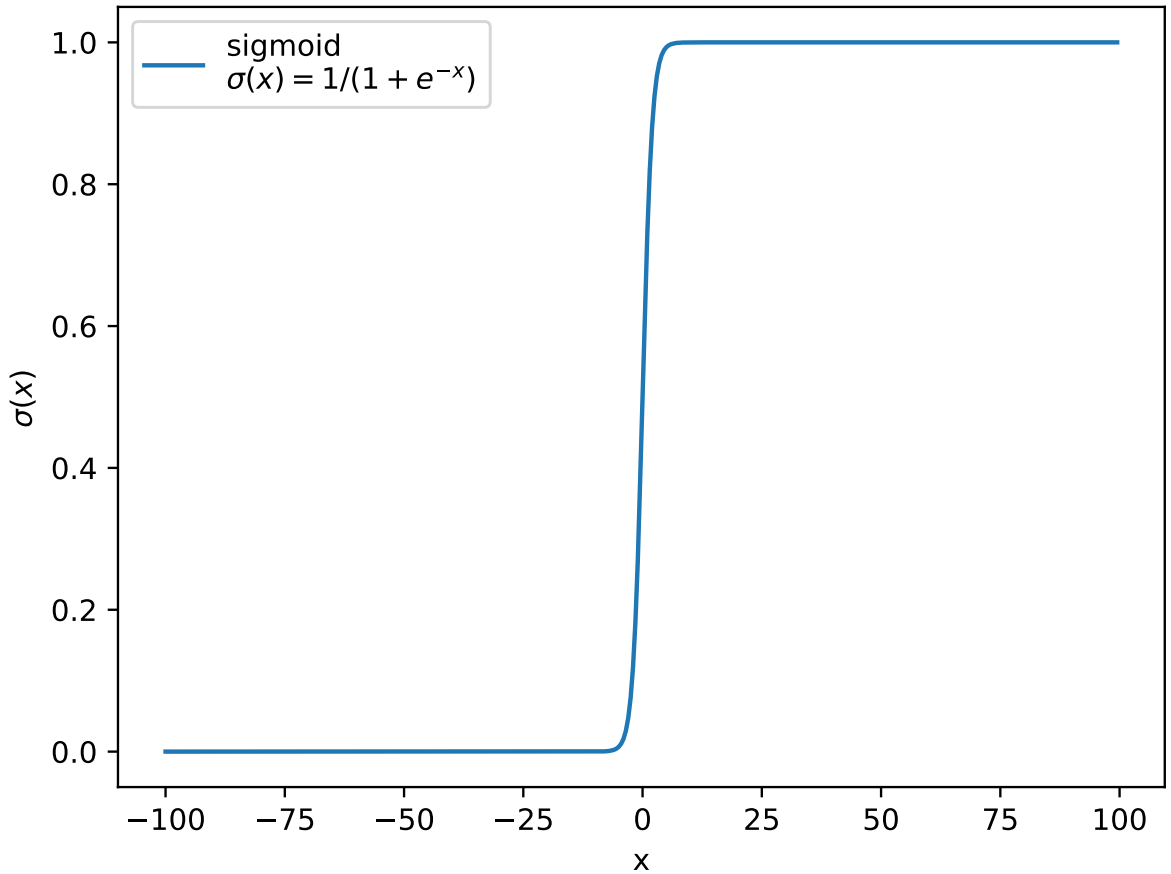
Sigmoid function



Sigmoid function



Sigmoid function



Notation with vectors

The sum $\sum_{i=1}^{d} x_i w_i$ can also be written this way :

$$x w^T \tag{2}$$

This means a **product** of **two matrices** (a vector is also a matrix : it is just a matrix with only one line or only one column):

- $x = (x_1, ..., x_d)$
- $$w^T = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{pmatrix}$$

Matrices

A matrix is an array used to store data. It has **lines** and **columns**

$$A = \begin{pmatrix} 1 & 4 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{pmatrix}$$

A_{ij} means the element at line i and column j.

- $A_{12} = 4$
- $A_{31} = 0$
- $A_{33} = 1$

Matricial multiplication

- If matrix A has p columns and matrix B has p lines, the product AB of the two matrices is defined as:

$$AB_{ij} = \sum_{k=1}^{p} A_{ik} B_{kj} \tag{3}$$

- `https://en.wikipedia.org/wiki/Matrix_multiplication`
- It is often more convenient and compact to write computations as operations on vectors and matrices when possible.

Examples

$$\begin{pmatrix} 1 & 4 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} = ?$$

Examples

$$\begin{pmatrix} 1 & 4 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

Examples

$$\begin{pmatrix} 1 & 4 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = ?$$

Examples

$$\begin{pmatrix} 1 & 4 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 4 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{pmatrix}$$

Examples

$$\begin{pmatrix} 1 & 4 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 1 & 0 & 0 \end{pmatrix} = ?$$

Examples

$$\begin{pmatrix} 1 & 4 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 1 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 8 & 0 \\ 2 & 2 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

Examples

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

What is A^n ?

Examples

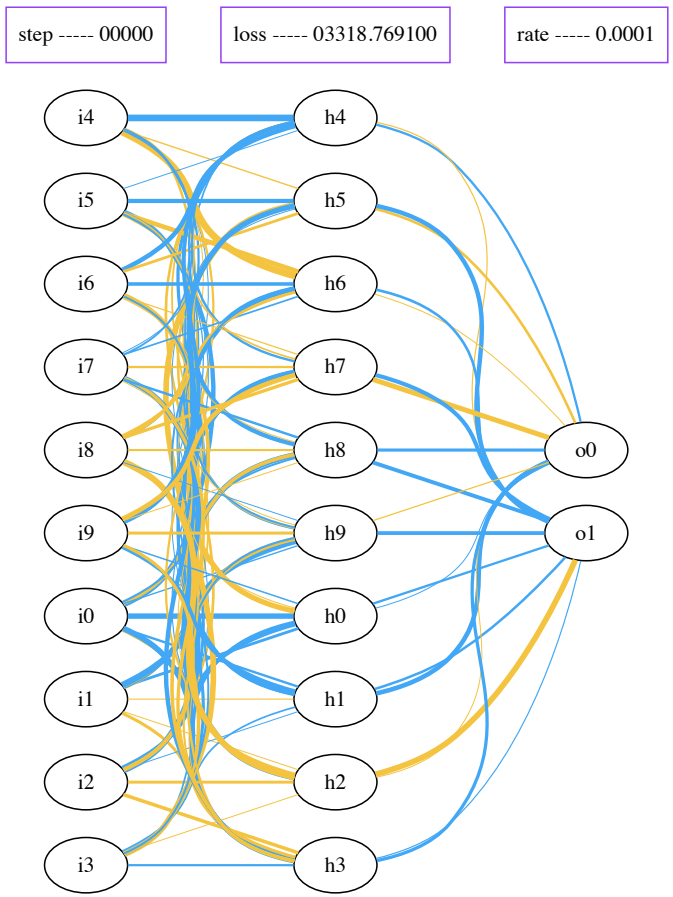
- $x = (x_1, ..., x_d)$
- $w = (w_1, ..., w_d)$

$$xw^T = \sum_{i=1}^{d} x_i w_i \tag{4}$$

Neural networks

- $\sigma(xw^T)$ allows us to compute the output of a **single neuron**
- But we will often have **several neurons** outputting a result.
- These neurons are organized in a network called neural network.

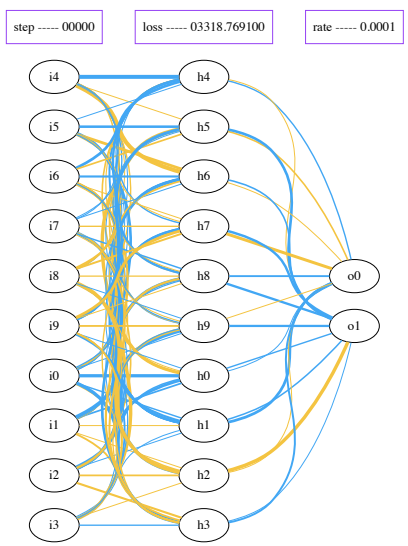
- Let $w = [w_{ij}]$ be the matrices of weights between neuron i of the left layer and neuron j of the middle layer.



Matrices and neural networks

- let $w = [w_{ij}]$ be the matrices of weights between neuron i of the left layer and neuron j of the middle layer.
- the input b_j of the hidden layer as a function of the outputs x_k of the input layer writes:

$$b_j = \sum_{k=1}^{d} x_k w_{kj} \tag{5}$$

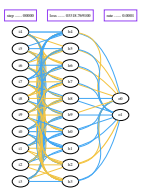


- let $w = [w_{ij}]$ be the matrices of weights between neuron i of the left layer and neuron j of the middle layer.

$$b_j = \sum_{k=1}^{d} x_k w_{kj} \tag{6}$$

- With $b = (b_1, ..., b_m)$ and $x = (x_1, .., x_d)$, we have

$$b = xw \tag{7}$$



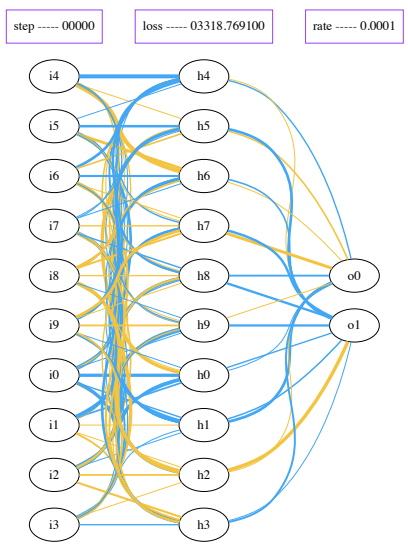
- The same notation generalizes to an arbitraty number of input samples $x_i, i \in [1, n]$!

Layers

- We now know how to compute the input b of a layer as a function of the output x of the previous layer

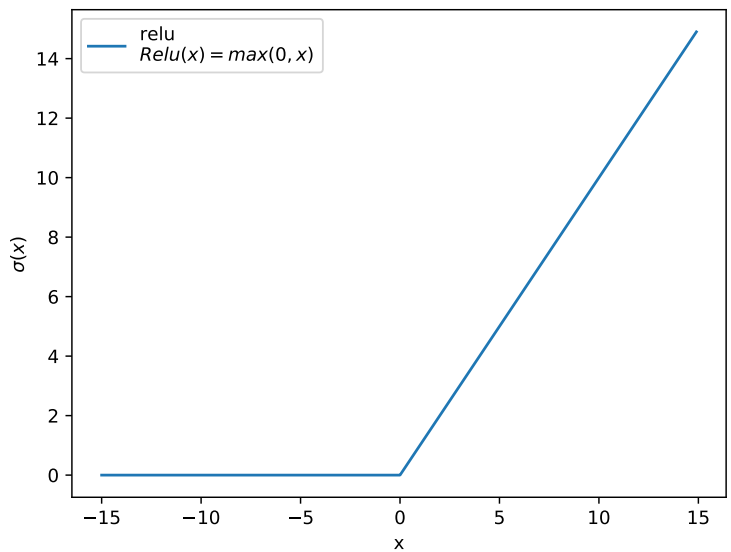
$$b = xw \tag{8}$$

- Applying this rule **and** the non linearity σ, we can coompute the **forward propagation** of a neural network.

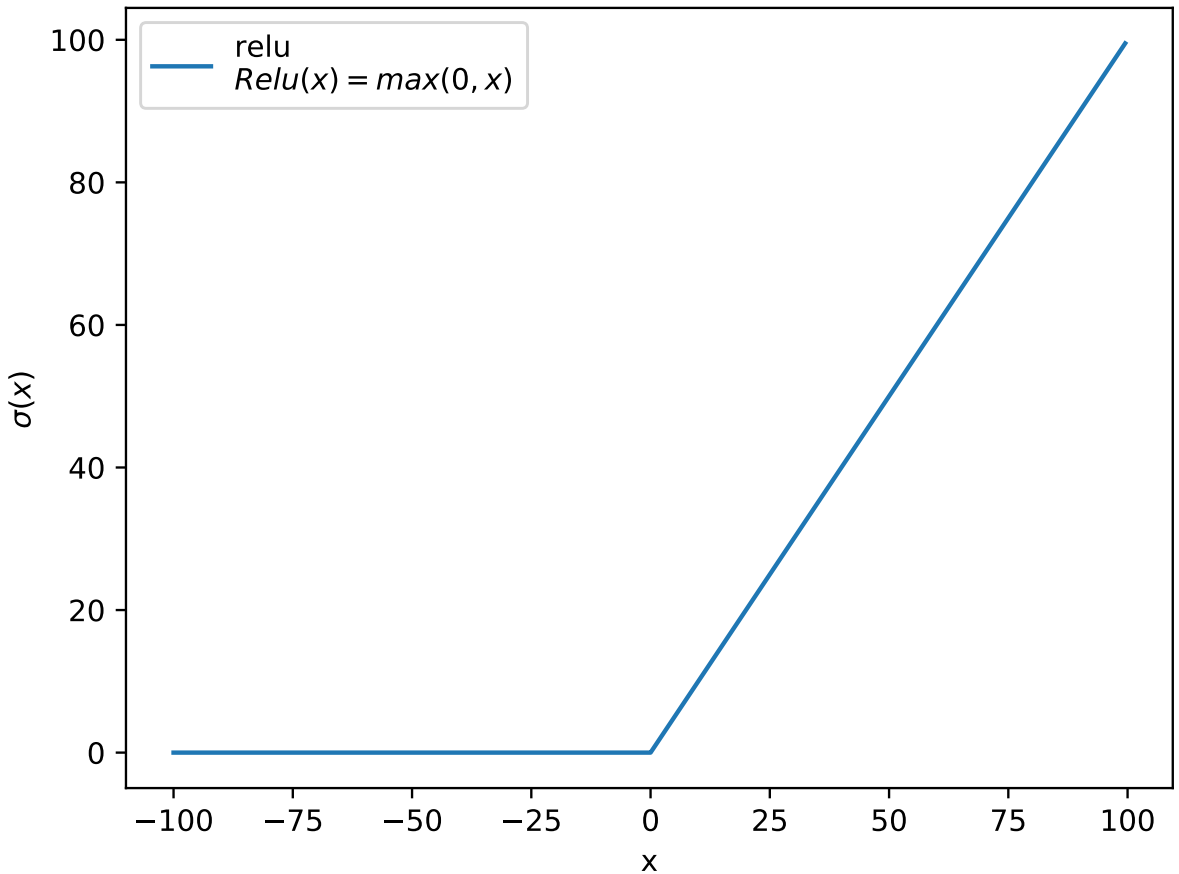


Layers and forward propagation

- We will use the **numpy** library to do so.
- We will use de ReLu as the non linearity.

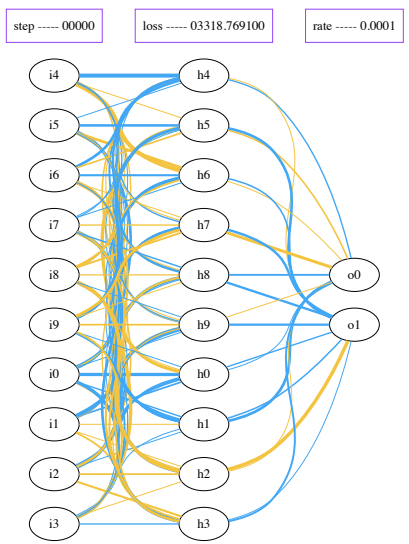


Relu function



Optimization

- The parameters are the **weights** w_1 and w_2.
- In our examples we will use a network with three layers : input - hidden - output.



Gradients in a neural network

- Example of a gradient

$$\frac{\partial L}{\partial w1} = h^T 2(y_{\text{predicted}} - y_{truth}) \tag{9}$$

(where h is the output of the relu)

Backpropagation

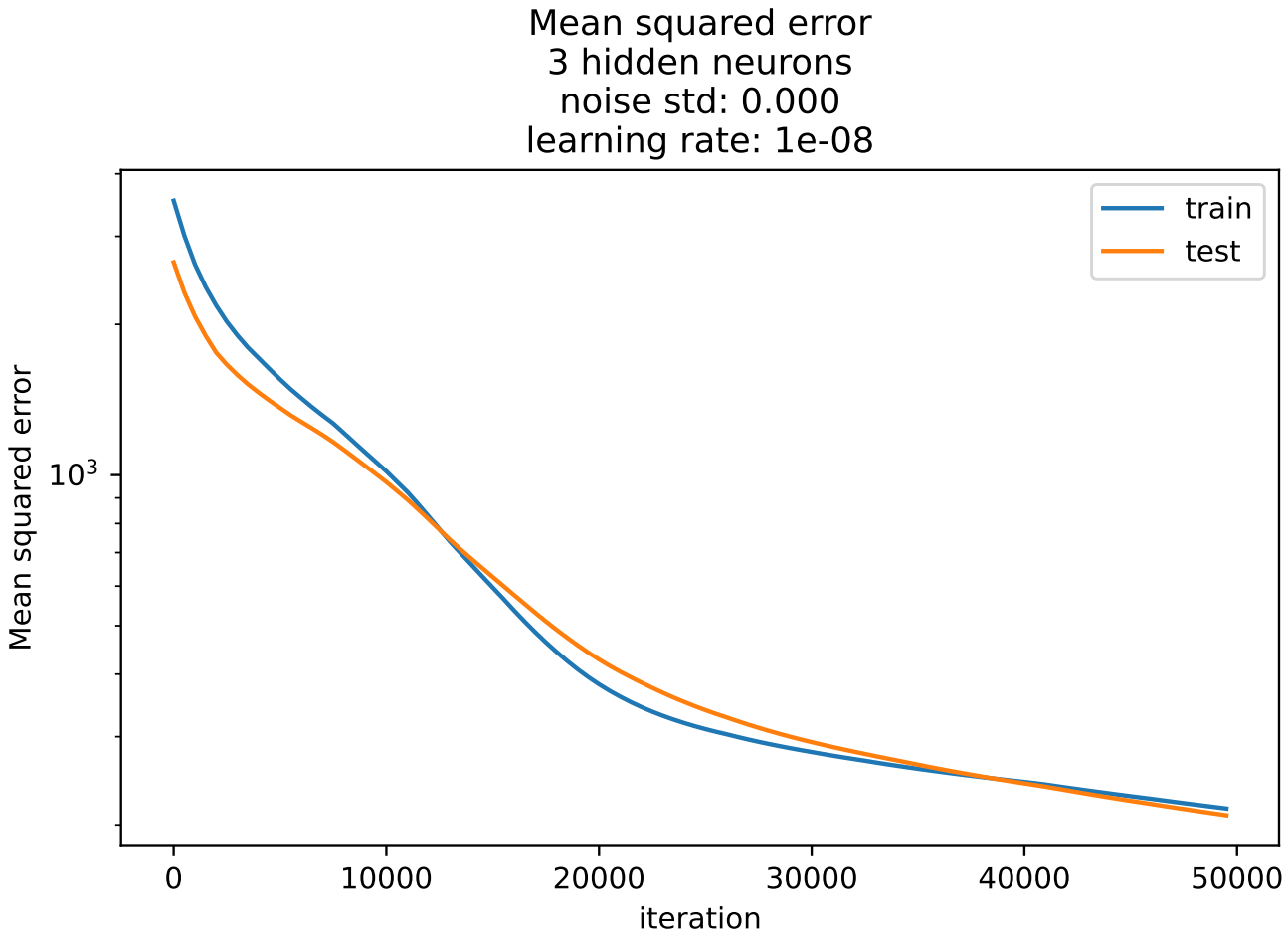
- By repeating the same process we can also compute the gradient with respect to $w2$.
- This is called **backpropagation**.
- Knowing the gradient, we can **update the network parameters**.

Learning toy data

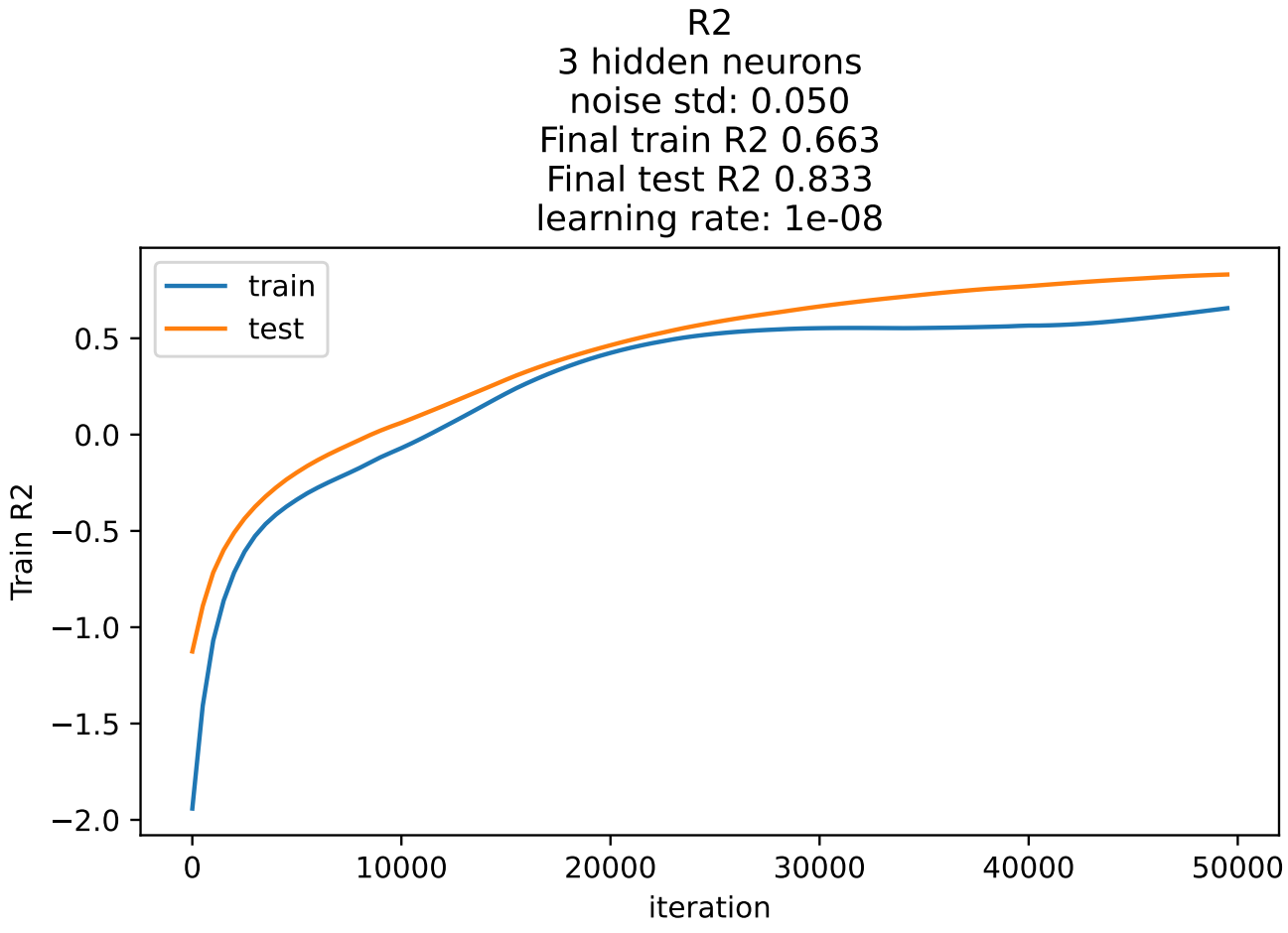
Exercice 1 :

- **cd 2_neural_networks/toy_data _numpy/**. Some toy data are generated in **create_data.py** by a function, + a noise.
- You can tune the standard deviation of the noise
- Use **main_train_neural_network.py** to predict these data by empirical risk minimization.
- You will need to experiment with the **hyperparameters** (defined at the top of the script).
- Make the network find a bad local minimum, make it explode (overflow), observe the network.

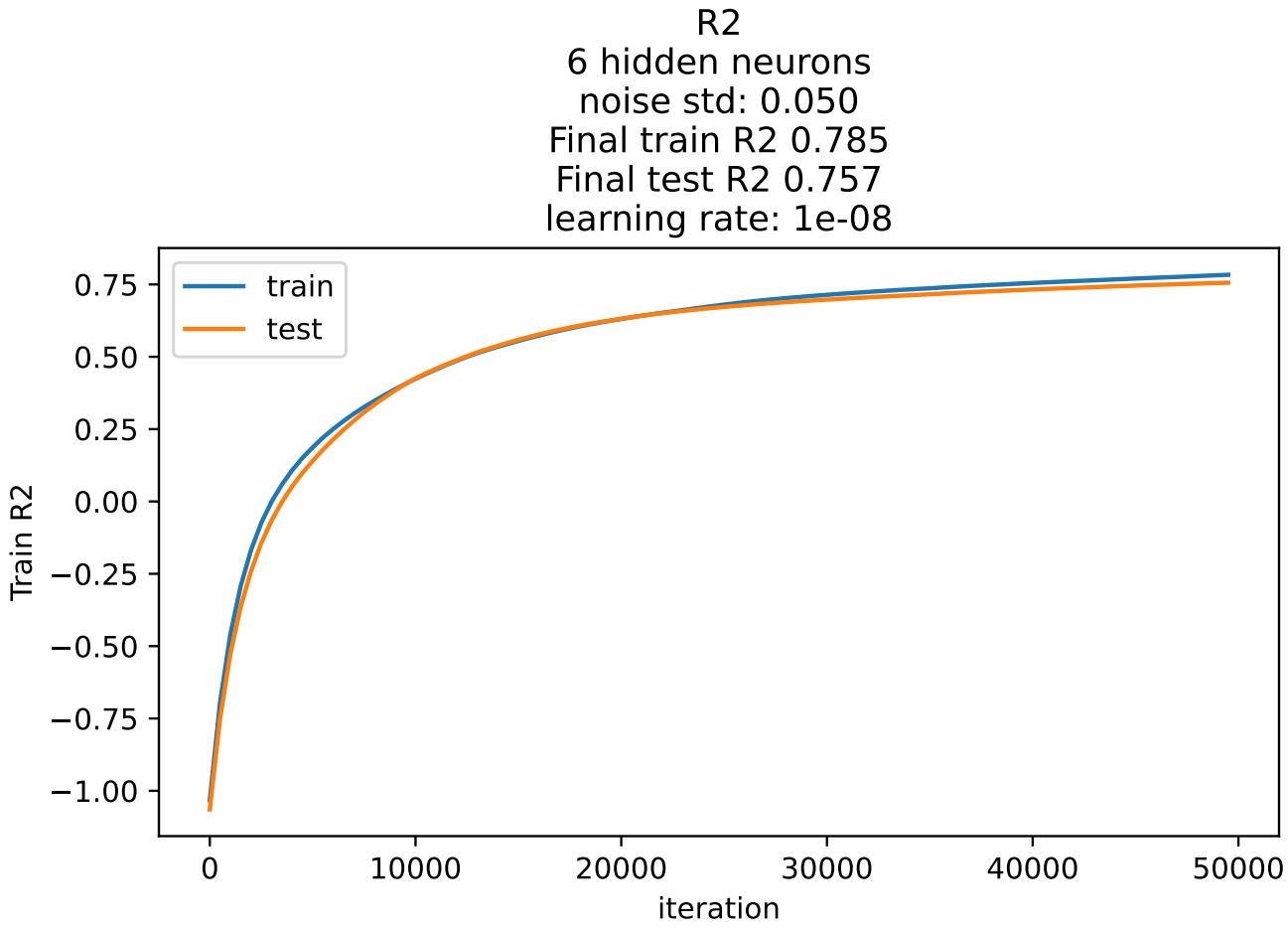
Learning curve



Learning curve



Learning curve



Plotting

Exercice 2 : **Observation of the network (optional)**

- Use **plot_net** so plot the evolution of the network
- You might need to use a smaller network otherwise it will be too long.

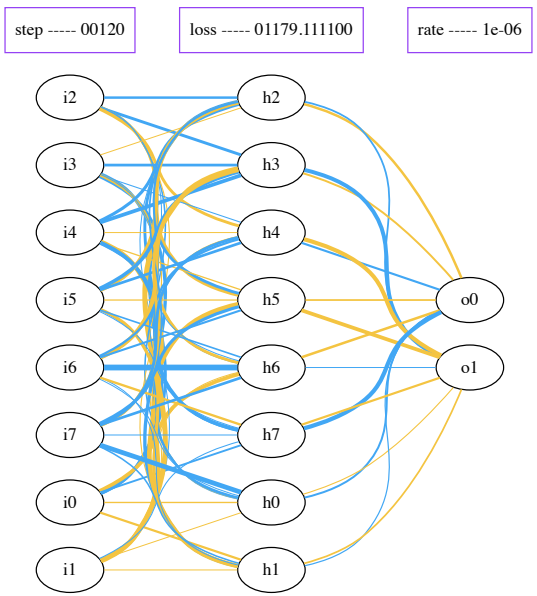
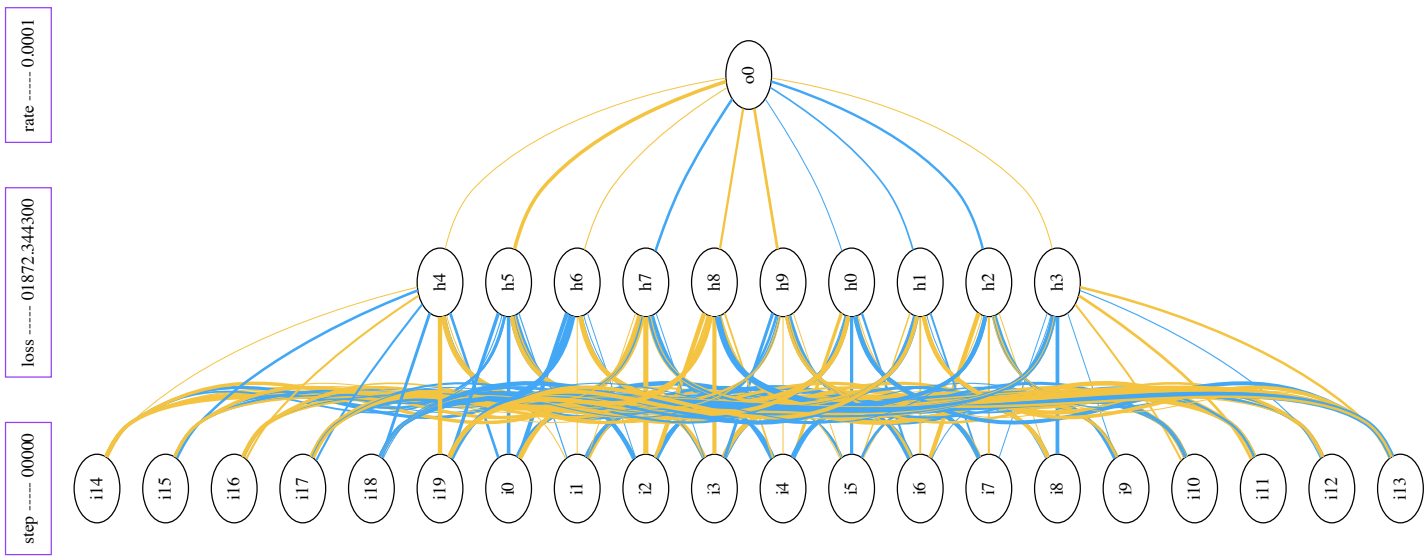
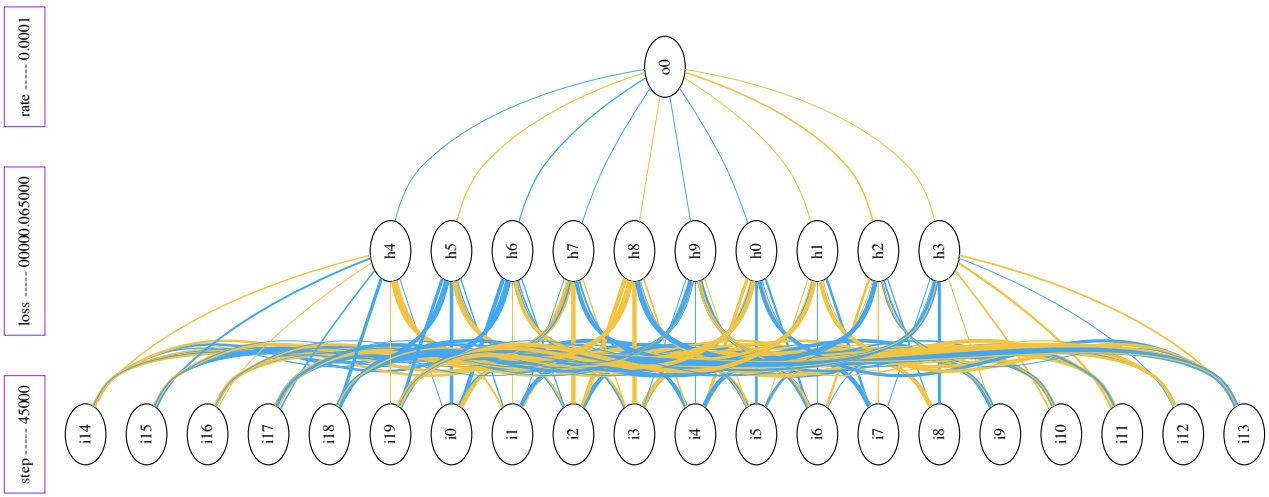


Figure:

Initial network



After 45000 steps



With libs

- We implemented a neural network manually with numpy, but when the networks are large or when we need to automate the seach for good parameters, it is more convenient to work with libraries such as :
  - pytorch
  - tensorflow
  - keras
  - theano

- There are many specific architectures and methods for neural networks:
  - in the number of neurons per layer
  - type of data processed
  - relationship between weights (shared weights)
  - number of hidden layers
  - recurrent neural networks RNN
  - convolutional neural networks CNN
  - graph convolutional networks GCN

Gradient descent

- Stochastic Gradient Descent (SGD)
- Mini-batch learning
- Batch learning

Other cost functions

- Until now we used the squared error cost function, but other cost functions also exist (especially for classification)
- The slowdown / vanishing gradient problem
- The Cross entropy is another possible cost function used for classification

MNIST

- **cd ../mnist**
- We will train a neural network to predicts digits in the MNIST database.
- With keras and tensorflow, we will achieve an accuracy of more than 97% in a few minutes for the classification.

Inputs

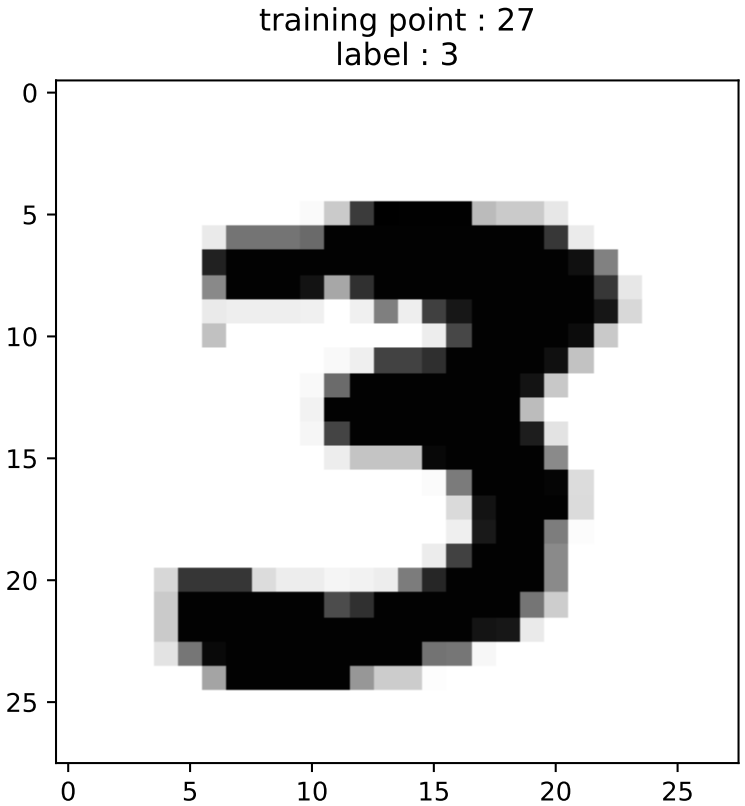


Figure: Datapoint 27

Inputs

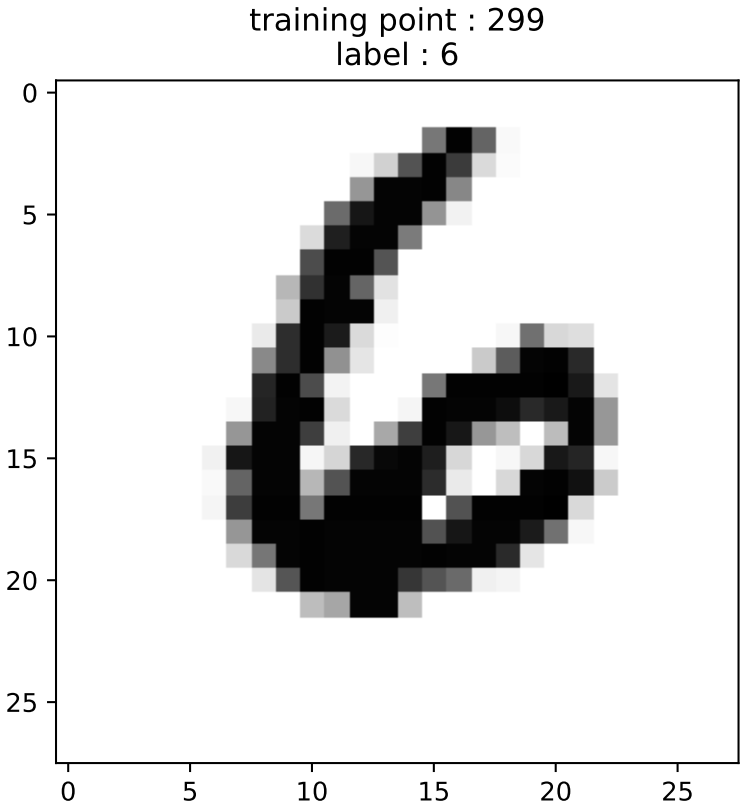


Figure: Datapoint 299

Inputs

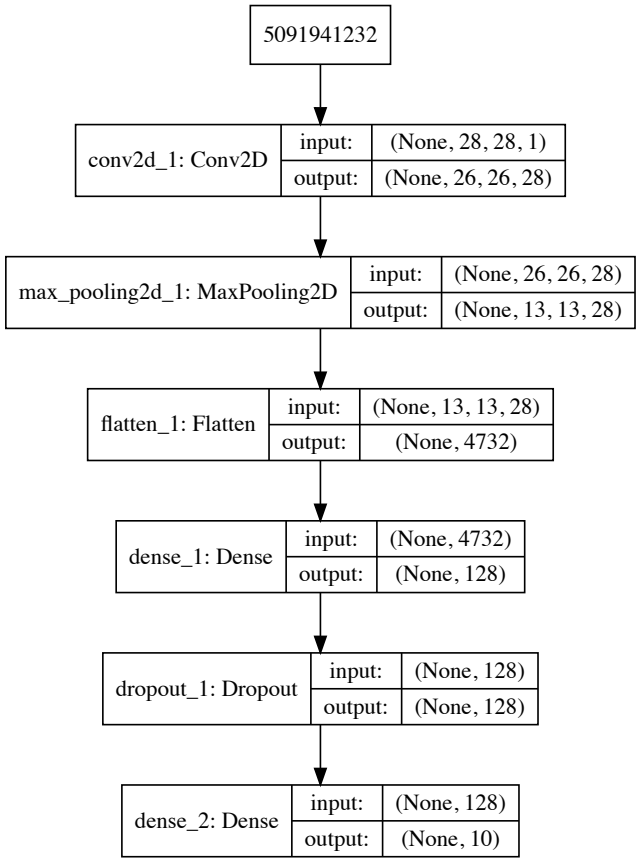


Figure: Network shape

Inputs

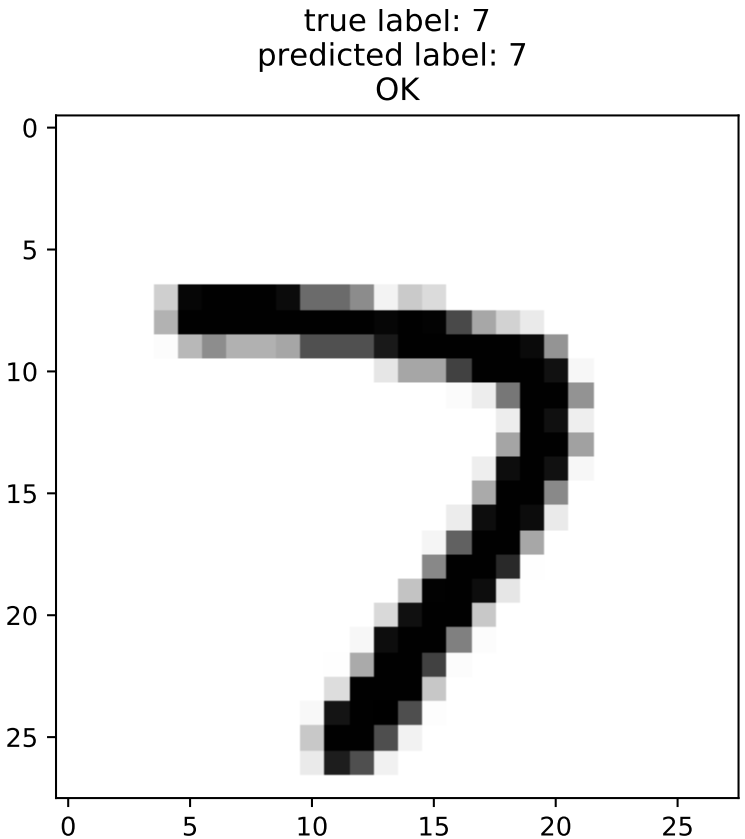


Figure: Predicion for point 17

Inputs

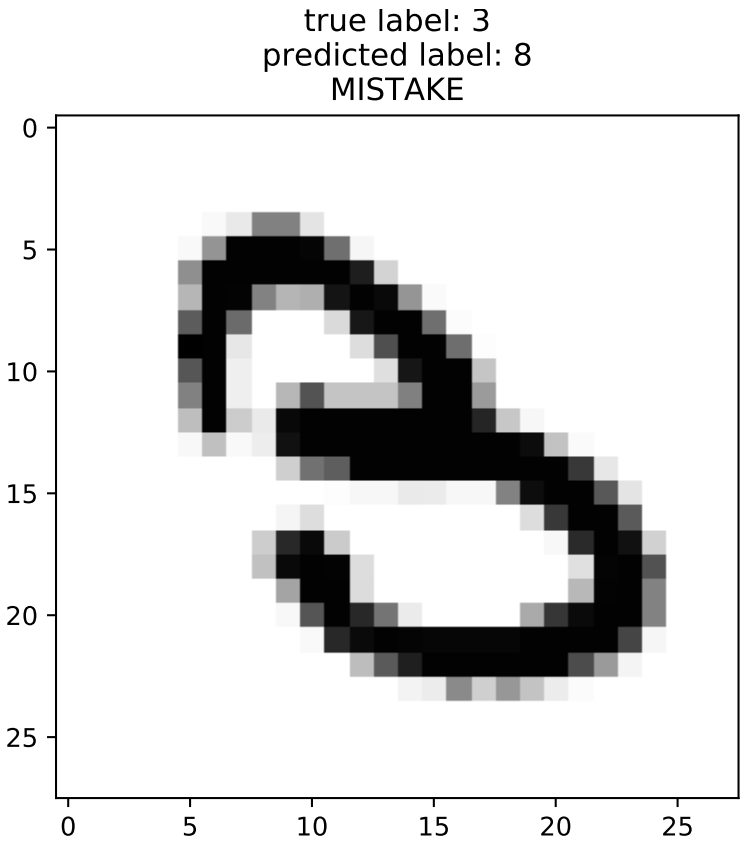
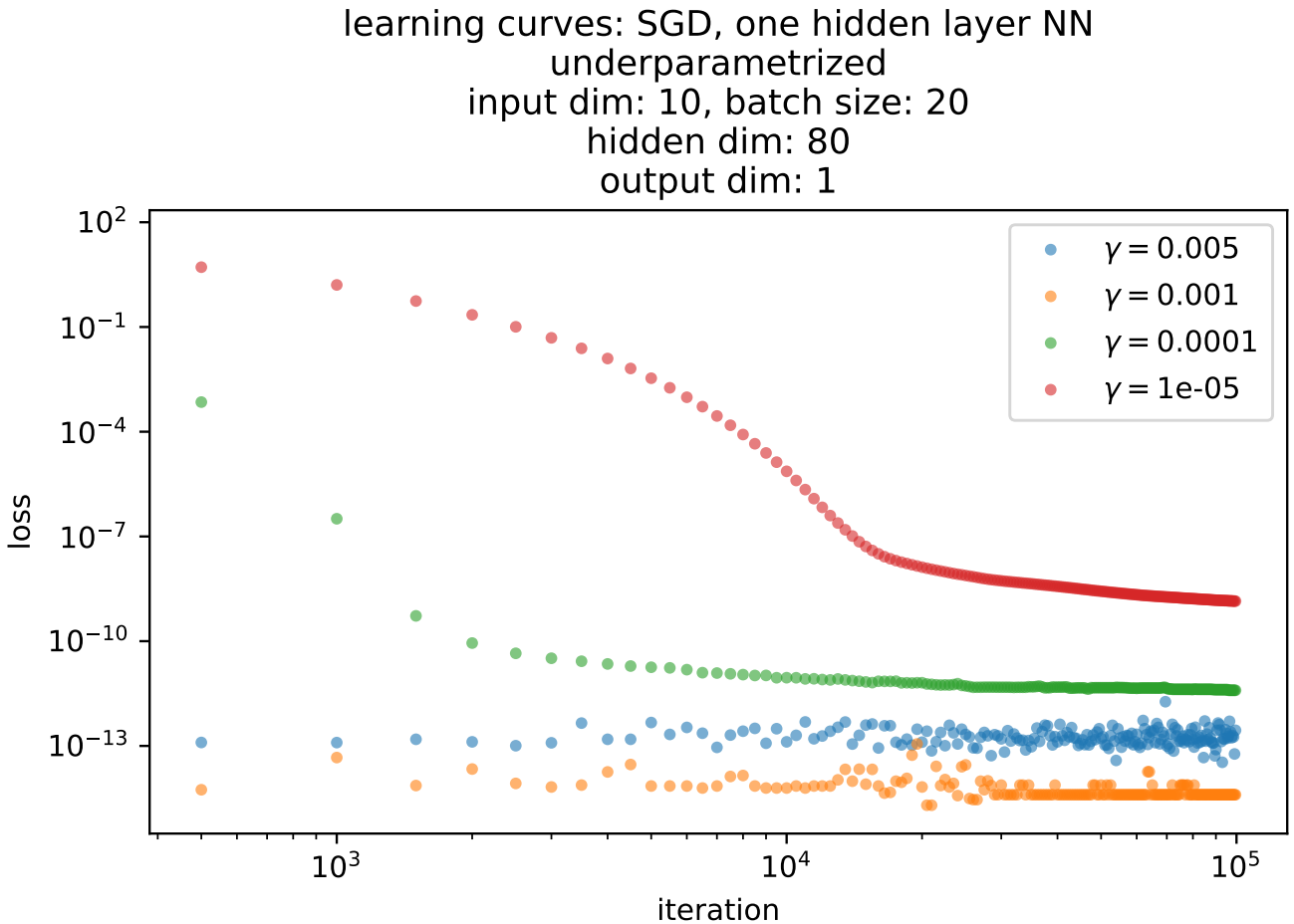


Figure: Prediction for point 18

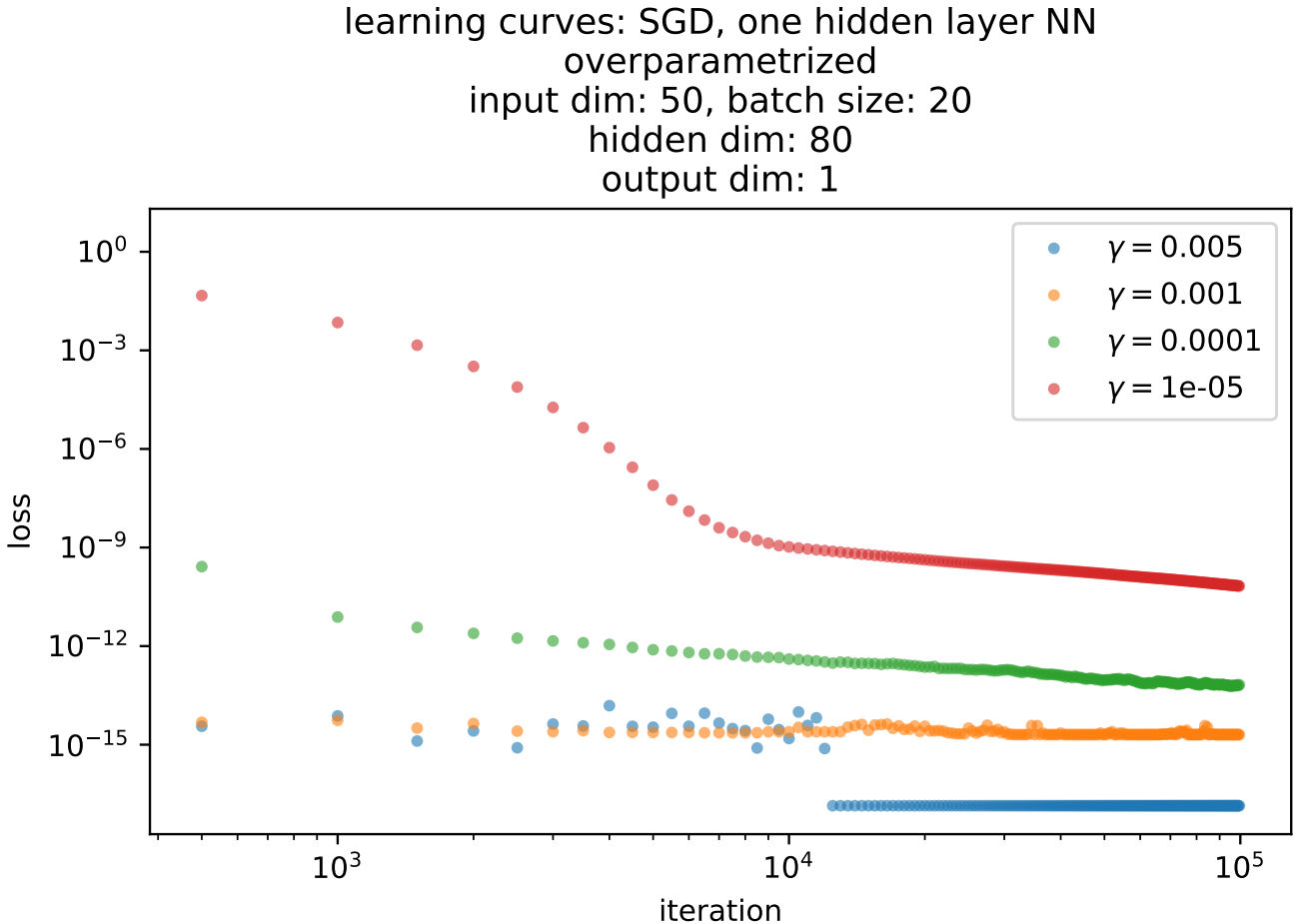
Underparametrization and overparametrization

Neural network have some very specific behaviors in some contexts.
`https://francisbach.com/rethinking-sgd-noise/`

In **day_2/2_neural_networks/overparametrization/**, we train neural networks in the overparametrized and in the underparametrized regime, this time using **torch**.



In the overparametrized regime, the SGD does not oscillate as much !



Double-descent phenomenon and overfitting

If optimized correctly, neural networks do not overfit easily !
This seems to be in contradiction with the classical machine learning theory.
`https://arxiv.org/pdf/1812.11118.pdf`

Profiling

`https://ui.perfetto.dev/` (profile obtained with **torch.profiler**). Profiling code running on GPUs has some specificities, compared to CPUs (using only cProfile is not enough).

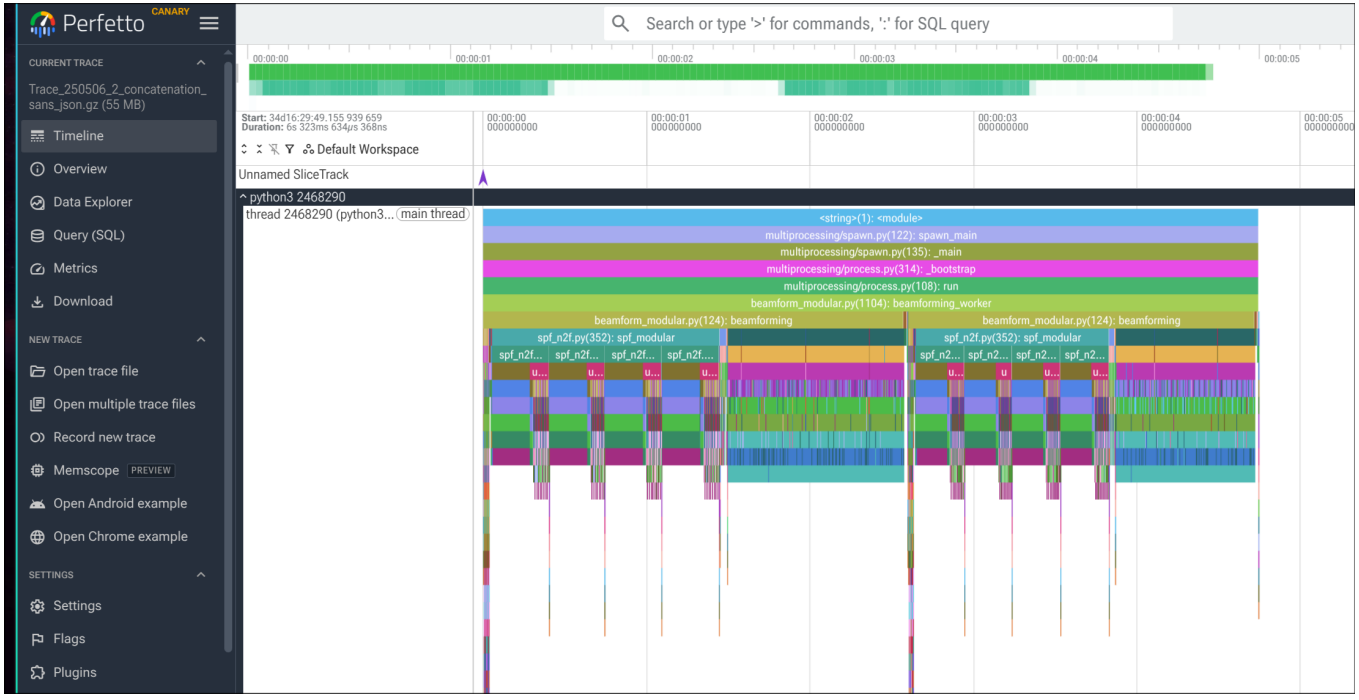


Figure: Profile of a **trace** obtained with the torch profilers.

Transfers

Transfers (or synchronisations) between CPU and GPU are a common cause of slowness:

- avoid using standard python functions called on tensors (tensor.sum() instead of sum(tensor))
- tensor.item(), tensor.cpu(), tensor.tolist() (sometimes used in console output)
- there are tools to look for GPU / CPU syncs in a code (for instance **torch.cuda.set_sync_debug_mode("warn")**)

For the project, you are encouraged to explore torch profilers and to create such visualizations and analysis.